

M-Colouring Backtracking

1 Problem Definition

M-Colouring Problem

Given a graph

$$G = (V, E)$$

and M available colours, assign one colour to each vertex so that no two adjacent vertices receive the same colour.

Let

$$V = \{1, 2, \dots, N\}, \quad C = \{1, 2, \dots, M\}.$$

The solution array is

$$x[v] = \text{colour assigned to vertex } v.$$

Colouring Constraint

For every edge $(u, v) \in E$,

$$x[u] \neq x[v].$$

If the colours are written as

$$\{R, G, B\},$$

then $M = 3$, and each $x[v]$ must be one of R , G , or B .

2 Three Versions

Version	Question
Decision	Can G be coloured using at most M colours?
Optimisation	What is the minimum number of colours needed?
Counting	How many valid M -colourings exist?

Chromatic Number

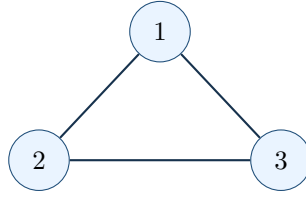
The minimum number of colours required to colour G is called the chromatic number and is denoted by

$$\chi(G).$$

3 Simple No-Solution Example

A triangle graph has three vertices and every pair of vertices is adjacent:

$$E = \{(1, 2), (2, 3), (1, 3)\}.$$



Try to colour this triangle with $M = 2$ colours, say R and G .

$$x[1] = R.$$

Since vertex 2 is adjacent to vertex 1, it must use the other colour:

$$x[2] = G.$$

Vertex 3 is adjacent to both 1 and 2, so it cannot use R and cannot use G . Therefore,

no valid 2-colouring exists.

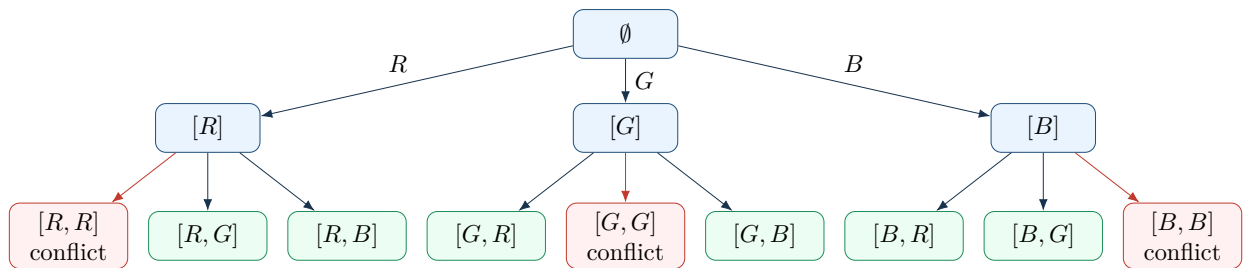
4 Backtracking State

At level k , the algorithm chooses a colour for vertex k . The partial solution is

$$(x[1], x[2], \dots, x[k]).$$

The state-space tree has at most M branches per level:

branching factor = M .



Example shown for two adjacent vertices: equal colours are pruned immediately.

5 Safety Test

A colour c is safe for vertex v if no already coloured neighbour of v has colour c .

Safety Condition

Let

$$N(v) = \{u \in V : (u, v) \in E\}$$

be the neighbour set of v . Colour c is safe for v when

$$x[u] \neq c \quad \text{for every already coloured } u \in N(v).$$

Safe

```
Safe(v, c):  
    for every vertex u adjacent to v:  
        if colour[u] == c:  
            return false  
  
    return true
```

6 Graph Colouring Algorithm

All M-Colourings

```
GraphColouring(vertex):  
    if vertex > numberOfVertices:  
        print colour[1..numberOfVertices]  
        return  
  
    for c = 1 to M:  
        if Safe(vertex, c):  
            colour[vertex] = c  
  
            GraphColouring(vertex + 1)  
  
            colour[vertex] = 0
```

The assignment

$$\text{colour}[\text{vertex}] = 0$$

removes the colour during backtracking so that the next branch starts from the correct partial colouring.

7 Worked Example with Search Tree

Problem

Colour the path graph

$$1 - 2 - 3 - 4$$

using $M = 3$ colours R, G, B .

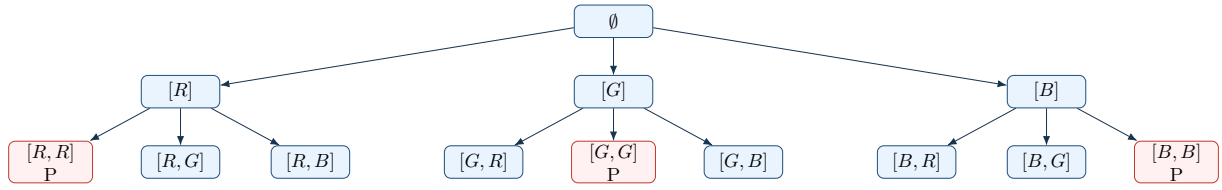


The algorithm colours vertices in the order 1, 2, 3, 4, trying colours in the order R, G, B .

Tree Notation

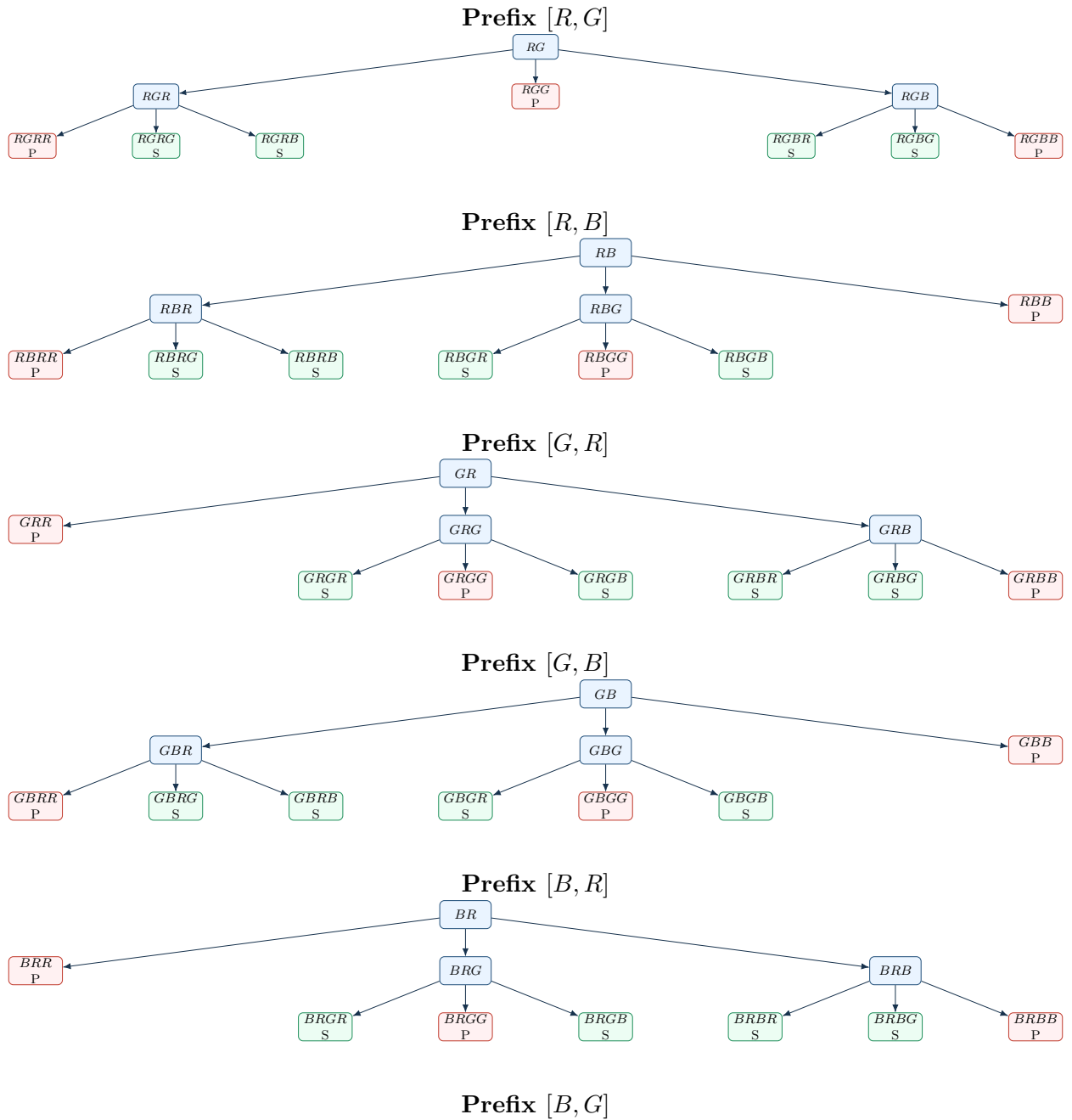
A red node marked P is pruned because it gives the same colour to two adjacent vertices. A green node marked S is a complete valid colouring.

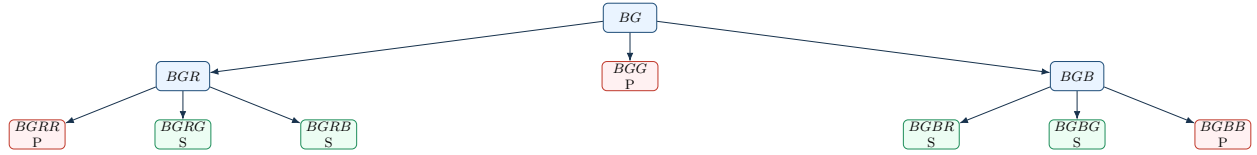
First Two Vertices



Remaining Subtrees

Each subtree below continues from one valid prefix after colouring vertices 1 and 2. The next two levels choose colours for vertices 3 and 4. To keep the tree readable, compact labels such as $RGRB$ mean the colour vector $[R, G, R, B]$.





Worked Example Result

The path graph has exactly

$$3 \cdot 2 \cdot 2 \cdot 2 = 24$$

valid 3-colourings. For example, $[R, G, R, G]$ and $[B, R, G, B]$ are valid complete paths.

8 First Solution vs All Solutions

Variant	Behaviour
First solution	Return immediately after one complete valid colouring is found.
All solutions	Print a complete colouring, backtrack, and continue searching.

9 Correctness Intuition

Every complete colouring corresponds to one path in the state-space tree because each vertex receives exactly one colour. The algorithm tries every colour for every vertex, but it recurses only when the partial colouring is safe.

1. If the algorithm prints a colouring, every edge has passed the safety test, so adjacent vertices have different colours.
2. If a valid colouring exists, none of its prefixes violates the safety test, so backtracking never prunes that valid path.

Conclusion

The algorithm prints exactly the valid M -colourings of the graph.

10 Complexity

In the worst case, each of the N vertices can try M colours:

$$M \cdot M \cdots M = M^N.$$

So the worst-case running time is

$$\mathcal{O}(M^N).$$

The recursion depth is N , and the colour array stores one value for each vertex:

$$\text{auxiliary space} = \mathcal{O}(N).$$

11 Final Review

M-Colouring Essentials

$G = (V, E), \quad |V| = N, \quad C = \{1, \dots, M\}.$

$x[v]$ = colour assigned to vertex v .

$(u, v) \in E \Rightarrow x[u] \neq x[v].$

Worst-case time = $\mathcal{O}(M^N)$, space = $\mathcal{O}(N)$.